

Análisis Predictivo: Maratón de Boston (2015-2017)

Aprendizaje Automático | Máster en Análisis de Datos Deportivos

Alba Martínez de la Hermosa Daniel Lopez Paredes
Alonso González Romero

6 de May, 2026

Table of contents

1	Introducción	4
	Contextualización y Motivación	4
2	Análisis Exploratorio de Datos	5
2.1	Importación de librerías	5
2.2	Carga de datos	5
2.3	Estructura del conjunto de datos	6
2.3.1	1. Datos Sociodemográficos y de Origen	6
2.3.2	2. Tiempos de Parciales (<i>Splits</i>)	7
2.3.3	3. Rendimiento y Clasificación Final	8
2.4	Integración y Consolidación del Conjunto de Datos	9
2.5	Preprocesamiento y Limpieza de Variables	10
2.6	Análisis de datos nulos	13
2.7	Incorporación de Variables Meteorológicas	14
2.8	Categorización por Nivel de Rendimiento	15
2.9	Definición de la Variable de Fatiga Extrema (<i>Hitting the Wall</i>)	17
3	Aprendizaje No Supervisado	19
3.1	Importación de librerías	19
3.2	Carga de datos	20
3.3	Clustering No Jerárquico: Identificación de Estrategias de Carrera (<i>Pacing</i>)	20
3.3.1	Fundamento Teórico: El Algoritmo K-Means	20
3.3.2	Metodología de Ejecución y Análisis	21
3.4	Clustering Jerárquico: Análisis Táctico de la Élite Femenina (2015)	22
3.4.1	Fundamento Teórico: Enfoque Aglomerativo (<i>Bottom-Up</i>)	22
3.4.2	Análisis Táctico	23
3.5	Clustering en Streaming: Simulación de Carrera en Vivo	28
4	Aprendizaje Supervisado	29
4.1	Importación de librerías	29
4.2	Carga de datos	30
4.3	Problema de clasificación. Detección de atletas que sufren en el muro.	30
4.3.1	K-Vecinos Más Cercanos (K-NN)	31
4.3.2	Árbol de Decisión	33
4.3.3	Aplicación del Algoritmo	34

4.3.4	Random Forest	35
4.3.5	Selección del mejor modelo y predicción en el conjunto de Validación . .	35
4.4	Problema de regresión. Predicción de tiempo final según sus datos hasta el parcial 35km	36

1 Introducción

Contextualización y Motivación

El maratón, una prueba de resistencia extrema de 42,195 kilómetros, representa uno de los retos de gestión energética y fisiológica más complejos en el ámbito del deporte de élite y amateur.

El presente trabajo se realiza usando como conjunto de datos los registros históricos del **Maratón de Boston** en sus ediciones de **2015, 2016 y 2017**.

Los principales **objetivos** de dicho trabajo son poner en práctica los algoritmos y modelos vistos en clase sobre *Aprendizaje Supervisado y No Supervisado*.

2 Análisis Exploratorio de Datos

En este primer capítulo se realizará un análisis exploratorio de datos (EDA) con el doble objetivo de comprender la estructura interna de los datasets actuales y de añadir variables externas (meteorológicas y temporales) que resulten significativas para el posterior entrenamiento de nuestros modelos.

Aunque el núcleo de este trabajo es el modelado predictivo, la fase de exploración y preprocesamiento representa el pilar fundamental que garantiza la calidad y robustez de cualquier sistema de aprendizaje automático.

2.1 Importación de librerías

Para comenzar, cargamos el ecosistema básico de librerías en Python para realizar dicho trabajo de exploración y preprocesamiento.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2.2 Carga de datos

Disponemos de tres archivos independientes en formato CSV que contienen los resultados oficiales de la Maratón de Boston para las ediciones de 2015, 2016 y 2017. Procedemos a su lectura para poder posteriormente empezar a realizar los diferentes análisis.

```
# Si no os deja cargarlo al trabajar (no al hacer quarto render), en la consola importar os y
# import os
# os.getcwd (y si no está en notebook, poneis en la consola os.chdir("notebooks"))

df_2015 = pd.read_csv('../data/raw/results2015.csv')
df_2016 = pd.read_csv('../data/raw/results2016.csv')
df_2017 = pd.read_csv('../data/raw/results2017.csv')
```

2.3 Estructura del conjunto de datos

Una vez tenemos los tres conjuntos de datos, vamos a realizar un pequeño estudio de la estructura que tienen, para ello, vamos a comenzar revisando si tienen exactamente las mismas variables:

```
# Comprobamos si las columnas y su orden son idénticos
todo_igual = df_2015.columns.equals(df_2016.columns) and df_2016.columns.equals(df_2017.columns)

# Obtenemos el número de columnas de cada dataframe
num_cols_15 = len(df_2015.columns)
num_cols_16 = len(df_2016.columns)
num_cols_17 = len(df_2017.columns)

# Imprimimos el resultado con el formato que quieres
if todo_igual:
    print(f"¿Son exactamente iguales en columnas y orden?: Sí, y además tienen {num_cols_15} columnas.")
else:
    print(f"¿Son exactamente iguales en columnas y orden?: No, tienen {num_cols_15}, {num_cols_16} y {num_cols_17} columnas.")
```

¿Son exactamente iguales en columnas y orden?: Sí, y además tienen 29 columnas.

Por lo tanto, tenemos 3 conjuntos de datos iguales y que además tienen 29 columnas. A continuación, vamos a elaborar en una tabla markdown, el significado de cada una.

El dataset original de la Maratón de Boston (ediciones 2015, 2016 y 2017) cuenta con un conjunto de variables que combinan datos sociodemográficos, tiempos de paso parciales y clasificaciones finales. A continuación se detalla el significado de cada columna estructurada por categorías:

2.3.1 1. Datos Sociodemográficos y de Origen

Variable	Tipo de Dato	Descripción
bib	string	Número de dorsal del corredor. <i>Nota:</i> Puede contener caracteres no numéricos en atletas de élite o categorías especiales (ej. F1, W10).

Variable	Tipo de Dato	Descripción
name	Object	Nombre completo del atleta. Existen otras 2 columnas que dividen este nombre en nombre y apellidos
age	float64	Edad del corredor el día de la carrera.
gender	string	Género del corredor (M para masculino, F para femenino).
city	string	Ciudad de residencia registrada por el participante.
state	string	Estado o provincia de residencia (principalmente para corredores de EE. UU. y Canadá).
country_residence	string	Código de 3 letras que representa el país de origen del corredor (ej. USA, ESP, MEX).
contry_citizenship	string	Nacionalidad o país de ciudadanía del corredor (suele estar vacío si coincide con el país de residencia).

2.3.2 2. Tiempos de Parciales (*Splits*)

Estas columnas registran el tiempo transcurrido acumulado desde el pistoletazo de salida (o cruce de la línea de salida) hasta cada punto de control específico.

Variable	Tipo de Dato	Punto de Control	Distancia Acumulada
5K	string	Tiempo de paso por el kilómetro 5.	5,0 km
10K	string	Tiempo de paso por el kilómetro 10.	10,0 km

Variable	Tipo de Dato	Punto de Control	Distancia Acumulada
15K	string	Tiempo de paso por el kilómetro 15.	15,0 km
20K	string	Tiempo de paso por el kilómetro 20.	20,0 km
Half	string	Tiempo de paso por la Media Maratón.	21,097 km
25K	string	Tiempo de paso por el kilómetro 25.	25,0 km
30K	string	Tiempo de paso por el kilómetro 30.	30,0 km
35K	string	Tiempo de paso por el kilómetro 35.	35,0 km
40K	string	Tiempo de paso por el kilómetro 40.	40,0 km

2.3.3 3. Rendimiento y Clasificación Final

Variable	Tipo de Dato	Descripción
Pace	string	Ritmo medio por milla de toda la carrera en formato MM:SS.
projected_time	string	Tiempo final estimado o proyectado (generalmente equivalente al tiempo oficial).
official_time	Object	Tiempo neto oficial de meta (Chip Time). Representa el tiempo real de carrera desde que el atleta cruza la línea de salida hasta que pisa la de meta. (Variable
overall	Integer	Objetivo / Target Y). Posición en la clasificación general absoluta de la carrera.

Variable	Tipo de Dato	Descripción
gender_result	Integer	Posición en la clasificación general de su respectivo género.
division_result	Integer	Posición en la clasificación de su grupo de edad específico.
seconds	Integer	Segundos tardados en completar la maratón

2.4 Integración y Consolidación del Conjunto de Datos

Para poder entrenar y validar nuestros modelos de aprendizaje automático de forma global, es necesario unificar las tres fuentes de datos (`df_2015`, `df_2016` y `df_2017`) en un único dataframe consolidado.

Antes de realizar la concatenación, añadiremos la variable **Date** con la fecha exacta de realización de cada edición. La Maratón de Boston se celebra históricamente el tercer lunes de abril (Patriots' Day), correspondiendo a las siguientes fechas reales: * **Edición 2015:** 20 de abril de 2015 (2015-04-20) * **Edición 2016:** 18 de abril de 2016 (2016-04-18) * **Edición 2017:** 17 de abril de 2017 (2017-04-17)

Al añadir esta variable temporal no solo mantenemos la trazabilidad del año de origen de cada registro, sino que habilitamos la posibilidad de cruzar estos datos en el futuro con variables externas como la meteorología del día exacto del evento.

```
# 1. Añadimos la columna 'Date' a cada dataframe individual en formato string
df_2015['Date'] = '2015-04-20'
df_2016['Date'] = '2016-04-18'
df_2017['Date'] = '2017-04-17'

# 2. Concatenamos los tres conjuntos de datos verticalmente
# ignore_index=True evita que los índices repetidos (0, 1, 2...) de cada año colisionen
df_boston = pd.concat([df_2015, df_2016, df_2017], ignore_index=True)

# 3. Convertimos la columna 'Date' a tipo datetime oficial de Pandas
df_boston['Date'] = pd.to_datetime(df_boston['Date'])

# 4. Verificamos las dimensiones finales del dataset unificado
print(f"Dimensiones del dataset unificado final: {df_boston.shape}")
```

```
print(f"Número total de registros integrados: {df_boston.shape[0]} corredores.")
print(f"Número total de columnas: {df_boston.shape[1]}")
```

Dimensiones del dataset unificado final: (79640, 30)
 Número total de registros integrados: 79640 corredores.
 Número total de columnas: 30

Una vez realizada la unificación, verificamos los datos realizando un muestreo de algunas columnas:

```
# Seleccionamos una muestra representativa con columnas demográficas y la nueva variable Date
columnas_muestra = ['name', 'age', 'gender', 'country_residence', 'official_time', 'Date']
df_muestra = df_boston[columnas_muestra].sample(5, random_state=42)

# Imprimimos la muestra formateada en markdown limpio
print(df_muestra.to_markdown(index=False))
```

name	age	gender	country_residence	official_time	Date
Loaiza, David	48	M	USA	3:24:27	2017-04-17 00:00:00
Fulfer, Clay	52	M	USA	5:14:15	2016-04-18 00:00:00
Meadows, Karen L	52	F	USA	4:00:58	2016-04-18 00:00:00
Johnson, David T	32	M	USA	2:54:17	2015-04-20 00:00:00
Paine, A. R.	51	M	USA	3:53:52	2016-04-18 00:00:00

2.5 Preprocesamiento y Limpieza de Variables

Una vez consolidado nuestro conjunto de datos, es fundamental adecuar los tipos de datos de las variables y descartar aquellas columnas redundantes o irrelevantes (como identificadores de nombres alternativos, sufijos o formatos duplicados) antes de alimentar nuestros algoritmos de Aprendizaje Automático.

En esta fase de preprocesamiento realizaremos las siguientes acciones: 1. **Selección de características (*Feature Selection*)**: Conservaremos únicamente el subconjunto de variables demográficas, espaciales y de rendimiento necesarias, descartando variables de texto libre como

name o redundantes como seconds. 2. **Corrección de tipos de datos:** Convertiremos la edad (age) de formato decimal a entero, y las variables de texto repetitivo como gender y country_residence a tipo categórico. 3. **Conversión temporal (Formatos HH:MM:SS a segundos):** Transformaremos todos los parciales de paso (5k, 10k, etc.) y la variable objetivo (official_time) a segundos totales, permitiendo que los modelos matemáticos puedan procesar estas distancias como variables numéricas continuas. 4. **Tratamiento de la fecha:** Simplificaremos la columna Date para conservar únicamente la fecha del evento sin la hora.

```
# 1. Definimos el subconjunto de variables que queremos conservar
columnas_conservar = [
    'bib', 'display_name', 'age', 'gender', 'city', 'country_residence',
    '5k', '10k', '15k', '20k', 'half', '25k', '30k', '35k', '40k',
    'pace', 'official_time', 'overall', 'gender_result', 'division_result', 'Date'
]

# Filtramos el dataframe original
df_boston = df_boston[columnas_conservar].copy()

# 2. Transformación de tipos básicos
# Convertimos la edad a entero de forma segura (eliminando previamente nulos si existieran en el dataframe)
df_boston = df_boston.dropna(subset=['age'])
df_boston['age'] = df_boston['age'].astype(int)

# Convertimos a tipo categórico para optimizar memoria y compatibilidad
df_boston['gender'] = df_boston['gender'].astype('category')
df_boston['country_residence'] = df_boston['country_residence'].astype('category')

# Forzamos que las variables identificadoras queden como texto limpio
df_boston['bib'] = df_boston['bib'].astype(str)
df_boston['display_name'] = df_boston['display_name'].astype(str)
df_boston['city'] = df_boston['city'].astype(str)

# 3. Función robusta para convertir "HH:MM:SS" a segundos totales
def tiempo_a_segundos(valor):
    if pd.isna(valor):
        return np.nan

    val_str = str(valor).strip()
    # Tratamos nulos ocultos en el formato original de la maratón
    if val_str in ('-', '', 'NaN', 'nan'):
        return np.nan

    try:
```

```

    partes = val_str.split(':')
    if len(partes) == 3: # Formato HH:MM:SS
        return int(partes[0]) * 3600 + int(partes[1]) * 60 + int(partes[2])
    elif len(partes) == 2: # Formato MM:SS (por si acaso en algún parcial corto)
        return int(partes[0]) * 60 + int(partes[1])
    return float(val_str)
except ValueError:
    return np.nan

# Columnas temporales que pasaremos a segundos
columnas_tiempo = ['5k', '10k', '15k', '20k', 'half', '25k', '30k', '35k', '40k', 'official_time']

# Aplicamos la conversión a segundos
for col in columnas_tiempo:
    df_boston[col] = df_boston[col].apply(tiempo_a_segundos)

# 4. Modificar la variable Date para dejarla únicamente como fecha sin hora
df_boston['Date'] = pd.to_datetime(df_boston['Date']).dt.date

# 5. Comprobación final de los tipos de datos resultantes
print("Tipos de datos finales del dataset preprocesado:")
print(df_boston.dtypes)

```

Tipos de datos finales del dataset preprocesado:

bib	object
display_name	object
age	int64
gender	category
city	object
country_residence	category
5k	float64
10k	float64
15k	float64
20k	float64
half	float64
25k	float64
30k	float64
35k	float64
40k	float64
pace	object
official_time	int64
overall	float64

```

gender_result          float64
division_result        float64
Date                   object
dtype: object

```

2.6 Análisis de datos nulos

Una vez tenemos todos los datos enriquecidos con su formato ideal y las variables necesarias para realizar los modelos, ahora se va a revisar qué datos nulos tenemos y en qué variables. Para ello, comenzamos viendo en qué variables hay datos nulos y el número:

```

# 1. Calculamos las métricas de nulos por columna
total_registros = len(df_boston)
nulos_por_columna = df_boston.isnull().sum()
porcentaje_nulos = (nulos_por_columna / total_registros) * 100

# 2. Construimos un DataFrame resumen con los resultados
df_resumen_nulos = pd.DataFrame({
    'Variable': nulos_por_columna.index,
    'Valores Nulos': nulos_por_columna.values,
    'Total Registros': total_registros,
    'Porcentaje Nulos (%)': porcentaje_nulos.values
})

# Redondeamos el porcentaje a dos decimales para una visualización más limpia
df_resumen_nulos['Porcentaje Nulos (%)'] = df_resumen_nulos['Porcentaje Nulos (%)'].round(2)

# 3. FILTRADO: Nos quedamos únicamente con las variables que tengan más de 0 nulos
df_resumen_nulos = df_resumen_nulos[df_resumen_nulos['Valores Nulos'] > 0]

# 4. Imprimimos el resultado en formato Markdown utilizando tabulate de fondo
print(df_resumen_nulos.to_markdown(index=False))

```

Variable	Valores Nulos	Total Registros	Porcentaje Nulos (%)
5k	229	79638	0.29
10k	114	79638	0.14
15k	51	79638	0.06
20k	85	79638	0.11
half	62	79638	0.08
25k	81	79638	0.1

Variable	Valores Nulos	Total Registros	Porcentaje Nulos (%)
30k	88	79638	0.11
35k	86	79638	0.11
40k	76	79638	0.1

Observamos que hay un porcentaje ínfimo de datos nulos, es por ello que se decide eliminar aquellas filas que tengan algún dato nulo en los parciales.

2.7 Incorporación de Variables Meteorológicas

El rendimiento en una maratón está fuertemente condicionado por factores ambientales externos. Para que nuestros modelos de aprendizaje automático puedan capturar este impacto, añadiremos tres variables climatológicas clave asociadas a la fecha de cada edición:

1. **temp_mean_c**: Temperatura promedio durante las horas de la carrera (en grados Celsius).
2. **humidity_pct**: Porcentaje de humedad relativa promedio.
3. **wind_speed_kmh**: Velocidad promedio del viento (factor crítico en Boston debido a su recorrido lineal, donde el viento de cara actúa como un freno constante).

Las condiciones registradas para cada edición fueron: * **2015 (Frío, lluvia y viento de cara extremo)**: 8 °C, 93% de humedad, viento de 22 km/h. * **2016 (Clima templado/cálido y seco)**: 16 °C, 28% de humedad, viento de 13 km/h. * **2017 (Calor sofocante, adverso para larga distancia)**: 21 °C, 35% de humedad, viento de 11 km/h.

```
# 1. Definimos el diccionario con los datos climatológicos por fecha (Date)
# Usamos strings de fecha porque en el paso anterior convertimos 'Date' a tipo datetime/date
clima_datos = {
    '2015-04-20': {'temp_mean_c': 8.0, 'humidity_pct': 93, 'wind_speed_kmh': 22},
    '2016-04-18': {'temp_mean_c': 16.0, 'humidity_pct': 28, 'wind_speed_kmh': 13},
    '2017-04-17': {'temp_mean_c': 21.0, 'humidity_pct': 35, 'wind_speed_kmh': 11}
}

# Convertimos las llaves del diccionario a objetos datetime.date para asegurar que coincidan
import datetime
clima_datos_fechas = {datetime.datetime.strptime(k, "%Y-%m-%d").date(): v for k, v in clima_datos.items()}

# 2. Mapeamos las nuevas columnas en función de la fecha de la carrera
df_boston['temp_mean_c'] = df_boston['Date'].map(lambda x: clima_datos_fechas[x]['temp_mean_c'])
df_boston['humidity_pct'] = df_boston['Date'].map(lambda x: clima_datos_fechas[x]['humidity_pct'])
df_boston['wind_speed_kmh'] = df_boston['Date'].map(lambda x: clima_datos_fechas[x]['wind_speed_kmh'])
```

```
# 3. Comprobamos que se han añadido correctamente
```

```
print(df_boston[['Date', 'temp_mean_c', 'humidity_pct', 'wind_speed_kmh']].drop_duplicates())
```

Date	temp_mean_c	humidity_pct	wind_speed_kmh
2015-04-20	8	93	22
2016-04-18	16	28	13
2017-04-17	21	35	11

2.8 Categorización por Nivel de Rendimiento

Con el objetivo de enriquecer el análisis descriptivo y facilitar posteriores estrategias de muestreo estratificado o validación cruzada controlada, procedemos a crear una nueva variable categórica ordinal que clasifique el nivel de rendimiento de cada atleta.

Para definir los umbrales de rendimiento, adaptamos el criterio oficial de la World Athletics y los tiempos de corte de los cajones de salida de las *World Marathon Majors* (como Boston). Dado que los ritmos basales difieren fisiológicamente por sexo, establecemos reglas diferenciadas para hombres (M) y mujeres (F):

- **Élite:**
 - Hombres: < 2h 17m 00s (< 8220 segundos)
 - Mujeres: < 2h 43m 00s (< 9780 segundos)
- **Alto nivel:**
 - Hombres: [2h 17m 00s - 2h 30m 00s) (8220 a < 9000 segundos)
 - Mujeres: [2h 43m 00s - 3h 00m 00s) (9780 a < 10800 segundos)
- **Muy entrenado/a:**
 - Hombres: [2h 30m 00s - 3h 00m 00s) (9000 a < 10800 segundos)
 - Mujeres: [3h 00m 00s - 3h 30m 00s) (10800 a < 12600 segundos)
- **Moderadamente entrenado/a:**
 - Hombres: [3h 00m 00s - 3h 30m 00s) (10800 a < 12600 segundos)
 - Mujeres: [3h 30m 00s - 4h 00m 00s) (12600 a < 14400 segundos)
- **Principiante:**
 - Hombres: ≥ 3h 30m 00s (≥ 12600 segundos)
 - Mujeres: ≥ 4h 00m 00s (≥ 14400 segundos)

A continuación, implementamos una función de clasificación y la aplicamos sobre nuestro dataset unificado.

```
def asignar_categoria_rendimiento(row):
    genero = row['gender']
    tiempo = row['official_time']

    # Si por alguna razón el tiempo es nulo, devolvemos nulo
    if pd.isna(tiempo):
        return np.nan

    if genero == 'M':
        if tiempo < 8220:
            return 'Élite'
        elif tiempo < 9000:
            return 'Alto nivel'
        elif tiempo < 10800:
            return 'Muy entrenado/a'
        elif tiempo < 12600:
            return 'Moderadamente entrenado/a'
        else:
            return 'Principiante'

    elif genero == 'F':
        if tiempo < 9780:
            return 'Élite'
        elif tiempo < 10800:
            return 'Alto nivel'
        elif tiempo < 12600:
            return 'Muy entrenado/a'
        elif tiempo < 14400:
            return 'Moderadamente entrenado/a'
        else:
            return 'Principiante'

    else:
        # En caso de que exista otra etiqueta o nulo en género
        return 'No clasificado'

# 1. Aplicamos la función fila por fila
df_boston['athlete_level'] = df_boston.apply(asignar_categoria_rendimiento, axis=1)

# 2. Convertimos la columna a tipo categórica ordenada (de menor a mayor rendimiento)
orden_categorias = ['Principiante', 'Moderadamente entrenado/a', 'Muy entrenado/a', 'Alto ni
```



```
df_boston['athlete_level'] = pd.Categorical(df_boston['athlete_level'], categories=orden_cat)

# 3. Comprobamos la distribución de nuestros corredores por nivel de rendimiento
distribucion_niveles = df_boston['athlete_level'].value_counts().reindex(orden_categorias).to_dict()
distribucion_niveles.columns = ['Número de Corredores']
distribucion_niveles['Porcentaje (%)'] = (distribucion_niveles['Número de Corredores'] / len(df_boston))

# Imprimimos la tabla de distribución en formato Markdown
print(distribucion_niveles.to_markdown())
```

athlete_level	Número de Corredores	Porcentaje (%)
Principiante	39845	50.41
Moderadamente entrenado/a	28452	36
Muy entrenado/a	10262	12.98
Alto nivel	393	0.5
Élite	86	0.11

Por lo tanto, se observa que la lógica aplicada tiene sentido según el porcentaje de corredores que hay en cada umbral.

2.9 Definición de la Variable de Fatiga Extrema (*Hitting the Wall*)

Para habilitar un posterior enfoque de **Aprendizaje Supervisado de Clasificación**, definimos una nueva variable objetivo binaria denominada **hit_the_wall** (coloquialmente conocido como “petar”).

Establecemos el umbral de fatiga siguiendo los estándares de la literatura científica deportiva: un atleta ha sufrido una deceleración crítica si el tiempo empleado en completar la segunda mitad de la maratón es igual o superior al **120% (un 20% más lento)** del tiempo invertido en la primera mitad.

- **Primera Mitad (T_1):** Tiempo de paso por la media maratón (**half**).
- **Segunda Mitad (T_2):** Official Time – half.
- **Condición de “Petar”:** $T_2 \geq 1.20 \times T_1 \implies \text{hit_the_wall} = 1$.

```
# 1. Calculamos el tiempo de la primera y segunda mitad en segundos
t1 = df_boston['half']
t2 = df_boston['official_time'] - df_boston['half']
```

```

# 2. Calculamos el ratio de deceleración (T2 / T1)
deceleracion_ratio = t2 / t1

# 3. Creamos la variable binaria (1 si ratio >= 1.20, 0 en caso contrario)
df_boston['hit_the_wall'] = (deceleracion_ratio >= 1.20).astype(int)

# 4. Analizamos la distribución real de cuánta gente "peta" en nuestro dataset
distribucion_muro = df_boston['hit_the_wall'].value_counts().to_frame()
distribucion_muro.columns = ['Número de Corredores']
distribucion_muro['Porcentaje (%)'] = (distribucion_muro['Número de Corredores'] / len(df_boston)) * 100
distribucion_muro.index = ['Ritmo Constante (0)', 'Chocan contra el muro (1)']

# Imprimimos la tabla resultante
print(distribucion_muro.to_markdown())

```

	Número de Corredores	Porcentaje (%)
Ritmo Constante (0)	63060	79.78
Chocan contra el muro (1)	15978	20.22

3 Aprendizaje No Supervisado

En este capítulo se realizarán diversos estudios de Aprendizaje no Supervisado. Dada la naturaleza de los datos, se seguirán los siguientes enfoques:

1. **Clustering No Jerárquico** para descubrir las estrategias de carrera
2. **Clustering Jerárquico** para agrupar a corredores similares en cuanto a sus resultados en las 3 maratones
3. **Clustering en streaming** para seguir a los atletas de élite a través de cada parcial

3.1 Importación de librerías

Para comenzar, cargamos el ecosistema necesario:

```
import pandas as pd
import numpy as np

# El algoritmo para pacing
from sklearn.cluster import KMeans
# Las métricas para decidir si la 'k' es buena
from sklearn.metrics import silhouette_score
# Herramienta para escalar datos
from sklearn.preprocessing import StandardScaler
# Herramientas para hacer el jerárquico
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
# Herramientas para los mapas auto-organizados

import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

```
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 11
```

3.2 Carga de datos

Cargamos los datos resultantes tras haber hecho el análisis exploratorio de datos en la anterior sección.

```
# Si no os deja cargarlo al trabajar (no al hacer quarto render), en la consola importar os y
# import os
# os.getcwd (y si no está en notebook, poneis en la consola os.chdir("notebooks"))
df_boston = pd.read_parquet("../data/processed/boston_post_eda.parquet")
```

3.3 Clustering No Jerárquico: Identificación de Estrategias de Carrera (*Pacing*)

El objetivo de esta sección es descubrir y categorizar de forma automática las diferentes estrategias de ritmo (*pacing strategies*) empleadas por los corredores durante la maratón. Para ello, utilizaremos técnicas de aprendizaje no supervisado, buscando agrupar a los atletas según la distribución de su esfuerzo a lo largo de los 42,195 km, independientemente de su marca final absoluta.

3.3.1 Fundamento Teórico: El Algoritmo K-Means

Para llevar a cabo esta agrupación, implementaremos el algoritmo **K-Means** (*K-Medias*), uno de los métodos de clustering particional más robustos y utilizados.

A diferencia del clustering jerárquico, K-Means requiere que se defina a priori el número de grupos (k) que se desean encontrar. El algoritmo funciona de manera iterativa siguiendo este proceso:

1. **Inicialización:** Se seleccionan aleatoriamente k puntos en el espacio de características que actuarán como los centros iniciales de los grupos (centroides, μ_i).
2. **Asignación:** Se calcula la distancia euclidiana entre cada corredor y todos los centroides. Cada corredor se asigna al clúster cuyo centroide esté más cercano.
3. **Actualización:** Una vez asignados todos los puntos, se recalcula la posición de cada centroide moviéndolo a la media matemática de todos los corredores que pertenecen a ese grupo.

4. **Convergencia:** Los pasos 2 y 3 se repiten de forma iterativa hasta que los centroides dejan de moverse o se alcanza un número máximo de iteraciones.

Matemáticamente, K-Means busca minimizar la varianza intra-clúster, conocida como **Inercia** (J), que se define como la suma de las distancias al cuadrado de cada punto respecto a su centroide:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Donde k es el número de clústeres, C_i es el conjunto de puntos que pertenecen al clúster i , x es el punto de datos (la estrategia del corredor) y μ_i es el centroide del clúster.

3.3.2 Metodología de Ejecución y Análisis

Para que el algoritmo capture estrategias puras y no se sesgue por la velocidad absoluta del corredor, se debe seguir un riguroso proceso de ingeniería de características (*Feature Engineering*) y análisis posterior:

3.3.2.1 1. Transformación de los Tiempos (Normalización de Ritmos)

Los tiempos del dataset original son acumulativos. Para capturar el esfuerzo en cada tramo, se debe transformar la variable de paso calculando el **tiempo neto de cada segmento** (ej. $T_{10k} - T_{5k}$ para obtener el tiempo invertido exclusivamente entre el kilómetro 5 y el 10). Una vez calculados los tiempos netos por tramo, estos se deben dividir entre el tiempo final oficial (`official_time`). De este modo, cada variable representará el **porcentaje del tiempo total** que el atleta invirtió en ese parcial concreto.

3.3.2.2 2. Determinación del Número Óptimo de Clústeres (k)

Se iterará el algoritmo K-Means para distintos valores de k (por ejemplo, del 2 al 10). Para seleccionar el k óptimo que mejor defina las tipologías de corredores, se apoyará la decisión en dos métricas gráficas: * **El Método del Codo (*Elbow Method*)**: Analizando la curva de caída de la Inercia. * **El Análisis de la Silueta (*Silhouette Score*)**: Para medir la cohesión y separación real de los grupos formados.

3.3.2.3 3. Perfilado y Visualización de Estrategias

Una vez elegida la k óptima y asignados los grupos, se extraerán las coordenadas de los centroides finales. Se graficarán estos centroides en un gráfico de líneas superpuestas (Eje X: Segmento de carrera; Eje Y: Porcentaje de tiempo invertido). Esto permitirá nombrar a cada cluster.

3.3.2.4 4. Conclusiones y Cruce de Variables

Finalmente, se extraerá el valor de negocio de la agrupación realizando un análisis cruzado (*Cross-tabulation*). Se evaluará, por ejemplo, la relación entre cada cluster y la variable `hit_the_wall` o el porcentaje de atletas (sobre el total de ese grupo) que hay en cada clúster

3.4 Clustering Jerárquico: Análisis Táctico de la Élite Femenina (2015)

Mientras que el algoritmo K-Means nos permitió identificar macro tendencias de ritmo en la población general de corredores, el **Clustering Jerárquico** es la herramienta ideal para diseccionar grupos pequeños y específicos. En esta sección, abandonaremos el enfoque masivo para centrarnos en un ecosistema altamente competitivo: la categoría de Élite Femenina en la edición de 2015.

3.4.1 Fundamento Teórico: Enfoque Aglomerativo (*Bottom-Up*)

El algoritmo jerárquico aglomerativo asume inicialmente que cada observación (cada corredor) es un clúster independiente. A partir de ahí, el proceso iterativo es el siguiente:

1. **Cálculo de la Matriz de Distancias:** Se calcula la distancia entre todos los pares de clústeres existentes. La métrica más utilizada es la **Distancia Euclidiana** en un espacio de n dimensiones:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. **Fusión:** Se identifican los dos clústeres separados por la menor distancia y se unen para formar un nuevo clúster consolidado.
3. **Actualización:** Se recalcula la matriz de distancias para medir la lejanía entre el nuevo clúster formado y los clústeres restantes.
4. **Iteración:** Los pasos 2 y 3 se repiten hasta que todas las observaciones quedan englobadas en un único macro-clúster central.

3.4.1.1 Criterios de Enlace (*Linkage Criteria*)

El punto más crítico del algoritmo en el paso 3 es definir cómo se mide la distancia matemática entre dos grupos que contienen múltiples puntos. Para ello, existen diversos criterios de enlace. En nuestro análisis deportivo, evaluaremos principalmente:

- **Enlace Simple (*Single Linkage*):** Calcula la distancia entre los dos puntos más cercanos de cada clúster. Tiende a generar grupos alargados en forma de cadena.
- **Enlace Completo (*Complete Linkage*):** Calcula la distancia entre los dos puntos más alejados, favoreciendo la creación de grupos esféricos y compactos.

3.4.1.2 Representación Visual y Corte: El Dendrograma

El resultado de este proceso no es una simple asignación de etiquetas, sino una estructura de árbol denominada **Dendrograma**. En este gráfico, el eje vertical (altura) representa la distancia matemática a la que se han fusionado dos grupos.

3.4.2 Análisis Táctico

Para este estudio, aplicaremos esta técnica con el fin de identificar perfiles de consistencia temporal; es decir, agruparemos a los corredores basándonos en cómo ha evolucionado su rendimiento a lo largo de las ediciones de 2015, 2016 y 2017.

```
# 1. Filtramos: Nos quedamos con la edición 2015, mujeres y categoría Élite
# Convertimos Date a datetime para extraer el año de forma segura
filtro_2015 = pd.to_datetime(df_boston['Date']).dt.year == 2015
df_elite_f_2015 = df_boston[filtro_2015 & (df_boston['gender'] == 'F') & (df_boston['athlete'] != '')]

print(f"Total de corredoras en la Élite Femenina de 2015: {len(df_elite_f_2015)}")

# 2. Calculamos los tiempos netos de cada tramo (segmentos de 5km + sprint final)
df_elite_f_2015['net_0_5k'] = df_elite_f_2015['5k']
df_elite_f_2015['net_5_10k'] = df_elite_f_2015['10k'] - df_elite_f_2015['5k']
df_elite_f_2015['net_10_15k'] = df_elite_f_2015['15k'] - df_elite_f_2015['10k']
df_elite_f_2015['net_15_20k'] = df_elite_f_2015['20k'] - df_elite_f_2015['15k']
df_elite_f_2015['net_20_25k'] = df_elite_f_2015['25k'] - df_elite_f_2015['20k']
df_elite_f_2015['net_25_30k'] = df_elite_f_2015['30k'] - df_elite_f_2015['25k']
df_elite_f_2015['net_30_35k'] = df_elite_f_2015['35k'] - df_elite_f_2015['30k']
df_elite_f_2015['net_35_40k'] = df_elite_f_2015['40k'] - df_elite_f_2015['35k']
```

```
# 3. Normalizamos: Convertimos los netos a porcentajes del tiempo total
columnas_netas = ['net_0_5k', 'net_5_10k', 'net_10_15k', 'net_15_20k',
                  'net_20_25k', 'net_25_30k', 'net_30_35k', 'net_35_40k']

variables_clustering = []
for col in columnas_netas:
    nuevo_nombre = f"pct_{col}"
    df_elite_f_2015[nuevo_nombre] = df_elite_f_2015[col] / df_elite_f_2015['official_time']
    variables_clustering.append(nuevo_nombre)

# 4. Establecemos el nombre de la corredora como índice
# Esto es CRUCIAL para que SciPy ponga sus nombres reales al final del dendrograma
df_elite_f_2015.set_index('display_name', inplace=True)

# 5. Creamos el sub-dataframe definitivo (X) solo con las variables a agrupar
X_jerarquico = df_elite_f_2015[variables_clustering]

# Vista previa de las primeras 3 corredoras
print("\nMatriz de características tácticas (porcentajes) lista para SciPy:\n")
print(X_jerarquico.head(3).to_markdown())
```

Total de corredoras en la Élite Femenina de 2015: 16

Matriz de características tácticas (porcentajes) lista para SciPy:

display_name	pct_net_0_5k	pct_net_5_10k	pct_net_10_15k	pct_net_15_20k
Caroline Rotich	0.117194	0.120184	0.120184	0.118574
Mare Dibaba	0.117025	0.120129	0.120129	0.118634
Buzunesh Deba	0.116776	0.120106	0.120106	0.118498

Una vez tenemos las variables, vamos a realizar y graficar el clustering jerárquico.

Comenzamos calculando el clustering jerárquico utilizando un enlace simple:

```
# 1. Estandarizamos los datos para que todos los tramos de 5km pesen lo mismo
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_jerarquico)

# 2. Calculamos la matriz de enlaces (Clustering Jerárquico)
```

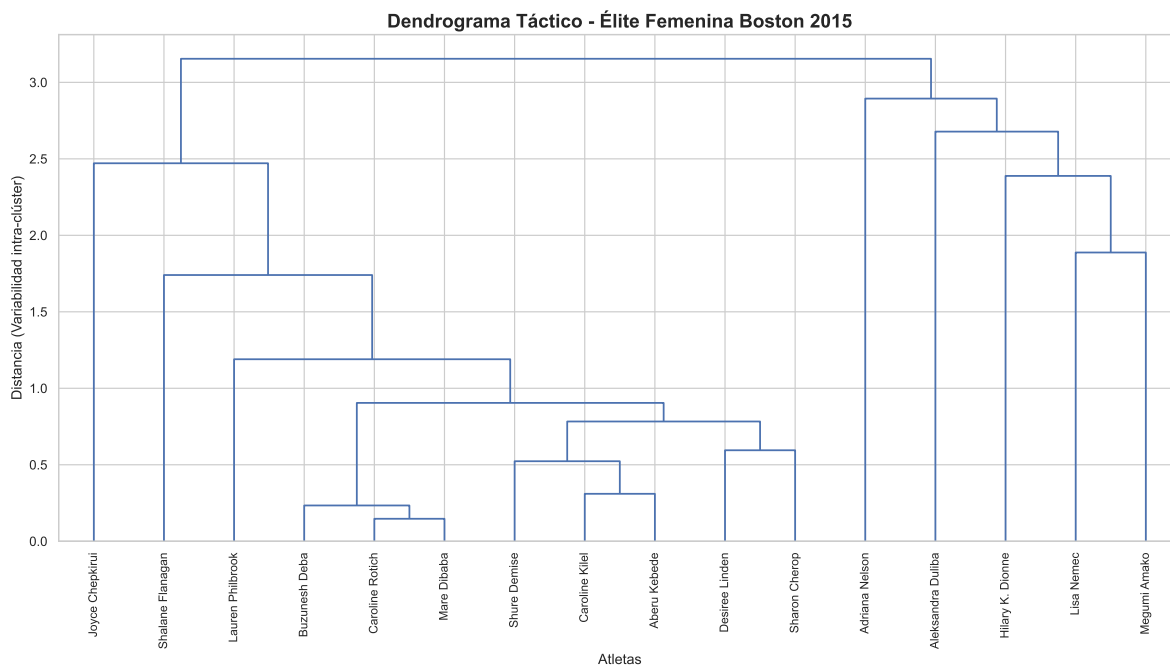


```
# La variable 'Z' contendrá toda la historia de cómo se han ido uniendo las corredoras
Z = linkage(X_scaled, method='single')

plt.figure(figsize=(14, 8))
plt.title("Dendrograma Táctico - Élite Femenina Boston 2015", fontsize=16, fontweight='bold')
plt.ylabel("Distancia (Variabilidad intra-clúster)", fontsize=12)
plt.xlabel("Atletas", fontsize=12)

# Dibujamos el dendrograma completo
dendro = dendrogram(
    Z,
    labels=X_jerarquico.index,
    leaf_rotation=90,
    leaf_font_size=10,
    color_threshold=0 # Lo ponemos a 0 para que de momento salga todo de un color hasta que c
)

plt.tight_layout()
plt.show()
```



Tras visualizar el dendrograma, observamos que se pueden sacar 2 grupos distintos. Los dividimos y vemos sus centroides para ver las estrategias

```

# 1. DEFINIR EL CORTE: Ajusta este valor numérico mirando el eje Y del dendrograma anterior
altura_corte = 3.0

# 2. Extraemos las etiquetas de los grupos
df_elite_f_2015['cluster_tactico'] = fcluster(Z, t=altura_corte, criterion='distance')

print("Distribución de atletas por grupo táctico: \n")
print(df_elite_f_2015['cluster_tactico'].value_counts().sort_index().to_markdown())

# 3. Calculamos las estrategias promedio (centroides)
centroides_jerarquico = df_elite_f_2015.groupby('cluster_tactico')[variables_clustering].mean()

# 4. Graficamos las líneas de ritmo de cada grupo
plt.figure(figsize=(12, 6))

# Limpiamos los nombres del Eje X para la leyenda
etiquetas_x = [col.replace('pct_net_', '').replace('_', '-') for col in variables_clustering]

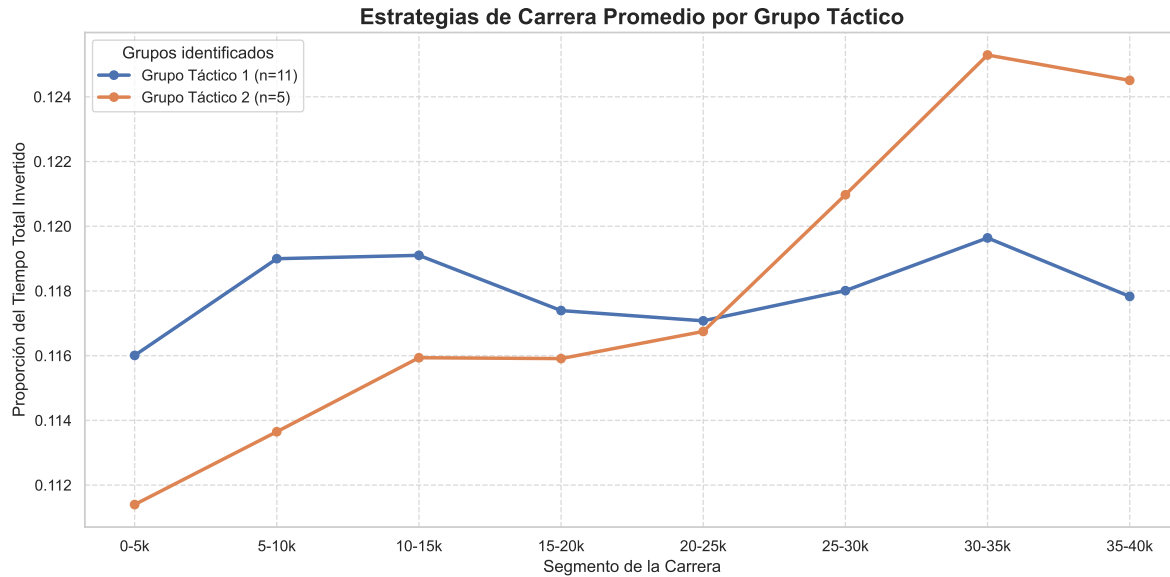
for cluster_id in centroides_jerarquico.index:
    plt.plot(
        etiquetas_x,
        centroides_jerarquico.loc[cluster_id],
        marker='o',
        linewidth=2.5,
        label=f'Grupo Táctico {cluster_id} (n={len(df_elite_f_2015[df_elite_f_2015["cluster_tactico"] == cluster_id])})
    )

plt.title("Estrategias de Carrera Promedio por Grupo Táctico", fontsize=16, fontweight='bold')
plt.xlabel("Segmento de la Carrera", fontsize=12)
plt.ylabel("Proporción del Tiempo Total Invertido", fontsize=12)
plt.legend(title="Grupos identificados")
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

```

Distribución de atletas por grupo táctico:

cluster_tactico	count
1	11
2	5



Podemos observar que el cluster 1 contiene 11 corredoras y es una estrategia mucho más constante que la del cluster 2, en la que es totalmente incremental.

El clustering jerárquico nos ha revelado la existencia de dos estrategias de carrera, pero... ¿cuál de ellas fue la más efectiva?

Para descubrirlo, cruzaremos las etiquetas de los grupos tácticos obtenidos con la posición final real de las atletas en su categoría (`division_result`). Calcularemos la posición media de cada pelotón e identificaremos en qué grupo se encontraba la ganadora absoluta de la prueba.

```
# 1. Agrupamos por clúster táctico y calculamos métricas sobre la posición
resumen_posiciones = df_elite_f_2015.groupby('cluster_tactico').agg(
    Num_Aletas=('division_result', 'count'),
    Posicion_Media=('division_result', 'mean'),
    Mejor_Posicion=('division_result', 'min')
).reset_index()
```

```
# 2. Añadimos una columna visual para identificar rápidamente dónde está la ganadora
resumen_posiciones['Ganadora_en_grupo'] = resumen_posiciones['Mejor_Posicion'].apply(lambda x: x)
```

```
# 3. Redondeamos los decimales para que la tabla quede limpia
resumen_posiciones = resumen_posiciones.round({'Posicion_Media': 1})
```

```
# 4. Renombramos columnas para la presentación
resumen_posiciones.rename(columns={
    'cluster_tactico': 'Grupo Táctico',
    'Num_Aletas': 'Nº Atletas',
    'Ganadora_en_grupo': 'Ganadora'
})
```

```

'Posicion_Media': 'Posición Final Media',
'Mejor_Posicion': 'Mejor Posición del Grupo',
'Ganadora_en_grupo': '¿Contiene a la Ganadora?'
}, inplace=True)

# 5. Imprimimos la tabla en formato Markdown
print("\n") # Salto de línea por seguridad para Pandoc
print(resumen_posiciones.to_markdown(index=False))
print("\n")

```

Grupo		Posición Final	Mejor Posición del	¿Contiene a la
Táctico	Nº Atletas	Media	Grupo	Ganadora?
1	11	6.5	1	Sí
2	5	13	11	No

Por lo tanto observamos que la estrategia constante (Cluster 1) es la mejor estrategia en cuanto a resultados obtenidos.

3.5 Clustering en Streaming: Simulación de Carrera en Vivo

La idea es cogerse a los atletas de élite femeninos y hacer un clustering en streaming al paso de cada parcial para ver cómo se hacen o deshacen los grupos de carrera.

4 Aprendizaje Supervisado

Este capítulo se divide en dos problemas distintos:

1. **Problema de clasificación:** Predecir si los corredores “petan” o no en el muro a través de la variable creada en el análisis exploratorio.
2. **Problema de regresión:** Predecir el tiempo final de los atletas al paso por los 35km. Cabe destacar que en la asignatura de *Análisis de Datos en el Deporte* se realizó un estudio con modelos de regresión y se observó que el parcial que mejores predicciones obtenía era el 35, por lo tanto, ahora vamos a probar varios modelos sobre este parcial para determinar cuál es mejor.

4.1 Importación de librerías

Comenzamos importando las librerías necesarias

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración visual
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

# =====
# 1. HERRAMIENTAS DE PREPARACIÓN DE DATOS (PARTICIONADO Y ESCALADO)
# =====

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# =====
# 2. MODELOS DE CLASIFICACIÓN (Para predecir "hit_the_wall")
# =====

from sklearn.neighbors import KNeighborsClassifier
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# =====
# 3. MODELOS DE REGRESIÓN (Para predecir el tiempo en meta)
# =====
from sklearn.linear_model import LinearRegression, Ridge, Lasso # Modelos Lineales Generalizados
from sklearn.tree import DecisionTreeRegressor
import xgboost as xgb # El rey del rendimiento

# =====
# 4. MÉTRICAS DE EVALUACIÓN (Para ponerles nota a los modelos)
# =====
# Para Clasificación:
from sklearn.metrics import accuracy_score, recall_score, classification_report, confusion_matrix
# Para Regresión:
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

```

4.2 Carga de datos

Realizamos la carga de datos.

```

# Si no os deja cargarlo al trabajar (no al hacer quarto render), en la consola importar os y
# import os
# os.getcwd (y si no está en notebook, poneis en la consola os.chdir("notebooks"))
df_boston = pd.read_parquet("../data/processed/boston_post_eda.parquet")

```

4.3 Problema de clasificación. Detección de atletas que sufren en el muro.

Para comenzar esta sección, vamos a realizar la división de nuestro conjunto de datos en Train-Test-Validation.

Para ello, la mejor opción será realizar un muestreo estratificado ya que hay un 79.78% de corredores que no chocan con el muro `hit_the_wall=0`.

A continuación, se hace el muestreo estratificado en una proporción 60-20-20 para cada conjunto. Además, se dejan en X las variables predictoras que pueden llegar a usarse (age, gender, country_residence, 5k, 10k, 15k, 20k, half, temp_mean_c, humidity_pct, wind_speed_kmh).

```

# 1. SELECCIÓN DE VARIABLES
features_todas = [
    'age', 'gender', 'country_residence',
    '5k', '10k', '15k', '20k', 'half',
    'temp_mean_c', 'humidity_pct', 'wind_speed_kmh'
]
target = 'hit_the_wall'

X = df_boston[features_todas]
y = df_boston[target]

# 3. DIVISIÓN ESTRATIFICADA (60/20/20)
# Primer corte: 60% Train / 40% Resto
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.40, random_state=42, stratify=y
)

# Segundo corte: Mitad del resto para Validación (20%) y mitad para Test (20%)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)

```

Una vez tenemos nuestros datos ya divididos, entrenaremos nuestros modelos en *Train*, los evaluaremos en *Test* y cuando tengamos todos los posibles modelos, elegiremos uno para probarlo en *Validation*.

Cabe destacar que evaluaremos el rendimiento de dichos modelos a través de las métricas:

- **Accuracy:** Una medida de cómo fallamos en general
- **Recall:** Ya que decirle a un atleta que no va a petar cuando en realidad peta es un error que puede costar muy caro, nuestro objetivo será maximizar dicha métrica, es decir, de todos los que realmente petan, ¿cuántos caza el modelo?

4.3.1 K-Vecinos Más Cercanos (K-NN)

El algoritmo de K-Vecinos Más Cercanos (K-NN) es un método de aprendizaje supervisado no paramétrico. A diferencia de otros algoritmos que construyen una ecuación matemática o un árbol de reglas durante el entrenamiento, K-NN es un algoritmo de tipo *lazy learning* (aprendizaje perezoso): no “aprende” un modelo en sí, sino que memoriza todo el conjunto de datos de entrenamiento para usarlo como mapa de referencia durante la predicción.

4.3.1.1 Fundamento del Algoritmo

Para clasificar a un nuevo corredor (es decir, predecir si sufrirá “El Muro” o no), el algoritmo sigue un proceso estrictamente geométrico:

1. **Cálculo de Distancias:** Mide la distancia matemática entre el nuevo corredor y todos los demás corredores presentes en el conjunto de entrenamiento. La métrica por defecto es la Distancia Euclidiana en un espacio n-dimensional:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. **Selección de la Vecindad (K):** Identifica a los K corredores del conjunto de entrenamiento que se encuentran a menor distancia del nuevo individuo. El valor K es un hiperparámetro crítico que se debe definir (generalmente un número impar para evitar empates, como 3, 5 o 7).
3. **Votación Mayoritaria:** El algoritmo observa la etiqueta real (`hit_the_wall`) de esos K vecinos más cercanos. La clase mayoritaria entre ellos será la predicción asignada al nuevo corredor.

4.3.1.2 Aplicación del Algoritmo

Una vez definido el marco teórico, aplicamos el algoritmo:

```
#1. Definimos las variables que vamos a usar:
variables_usar = ['age', '5k', '10k', '15k', '20k', 'half', 'temp_mean_c', 'humidity_pct', 'v
X_train_knn = X_train[variables_usar]
X_test_knn = X_test[variables_usar]
X_validation_knn = X_val[variables_usar]
#2. Las escalamos
scaler = StandardScaler()
X_train_knn = scaler.fit_transform(X_train_knn)
X_test_knn = scaler.transform(X_test_knn)
X_validation_knn = scaler.transform(X_validation_knn)
#3. Entrenamos el algoritmo
modelo_clasificacion_knn = KNeighborsClassifier(n_neighbors=1)
modelo_clasificacion_knn.fit(X_train_knn, y_train)
#4. Realizamos la predicción en test
predicciones_test = modelo_clasificacion_knn.predict(X_test_knn)

#5. Visualizamos la matriz de confusión
```



```

tabla_confusion = pd.crosstab(
    y_test,
    predicciones_test,
    rownames=['Realidad (0=No, 1=Sí)'],
    colnames=['Predicción (0=No, 1=Sí)']
)

print("=== MATRIZ DE CONFUSIÓN (K-NN) ===\n")
print(tabla_confusion)

print("=== EVALUACIÓN DEL MODELO KNN ===")
print(f"Accuracy Global: {accuracy_score(y_test, predicciones_test):.2%}")
print(f"Recall (Detección del Muro): {recall_score(y_test, predicciones_test, pos_label = 1)

```

```

=== MATRIZ DE CONFUSIÓN (K-NN) ===

Predicción (0=No, 1=Sí)      0      1
Realidad (0=No, 1=Sí)
0                             10655   1958
1                             2124   1071
=== EVALUACIÓN DEL MODELO KNN ===
Accuracy Global:              74.18%
Recall (Detección del Muro):  33.52%

```

Comenzamos estableciendo un modelo base utilizando K-NN. Como era de esperar ante un dataset desbalanceado (80/20), el algoritmo basado en distancias fue incapaz de identificar los colapsos, logrando un Recall máximo del 33% asumiendo un riesgo extremo de sobreajuste ($K = 1$). Esto justifica la necesidad de evolucionar hacia algoritmos basados en árboles que permitan un balanceo de pesos matemático.

4.3.2 Árbol de Decisión

El Árbol de Decisión (*Decision Tree*) es un algoritmo de aprendizaje supervisado no paramétrico que construye un modelo predictivo basado en una secuencia jerárquica de reglas lógicas. A diferencia de métodos basados en distancias (como K-NN) o en ecuaciones algebraicas (como la Regresión Lineal), el árbol divide el conjunto de datos iterativamente realizando preguntas binarias sobre las variables predictoras.

4.3.2.1 Funcionamiento y Criterio de Corte

En cada nodo, el algoritmo evalúa todas las características disponibles (edad, ritmos, país, etc.) y selecciona el punto de corte que mejor separa a los corredores según la variable objetivo (*hit_the_wall*). Para medir la calidad de esta separación, utiliza métricas matemáticas de “pureza”, siendo la más común el **Índice de Impureza de Gini**. El proceso se ramifica de forma descendente hasta alcanzar un criterio de parada definido por el analista (por ejemplo, una profundidad máxima, *max_depth*) para evitar el sobreajuste (*overfitting*).

4.3.3 Aplicación del Algoritmo

A continuación, se realiza el algoritmo. La idea será coger las variables categóricas como el país y binarizarlas, en vez de dejar todos los posibles países, ya que son muchos. De esta manera, logramos una regla lógica de si el corredor es o no de los estados unidos. Tras ello, entrenamos el algoritmo:

```
X_train_tree = X_train.copy()
X_test_tree = X_test.copy()
X_val_tree = X_val.copy()
# Binarizamos la variable país
X_train_tree['is_USA'] = (X_train_tree['country_residence'] == 'USA').astype(int)
X_test_tree['is_USA'] = (X_test_tree['country_residence'] == 'USA').astype(int)
X_val_tree['is_USA'] = (X_val_tree['country_residence'] == 'USA').astype(int)

# Convertimos Género (1 = Mujer, 0 = Hombre)
X_train_tree['is_Female'] = (X_train_tree['gender'] == 'F').astype(int)
X_test_tree['is_Female'] = (X_test_tree['gender'] == 'F').astype(int)
X_val_tree['is_Female'] = (X_val_tree['gender'] == 'F').astype(int)
columnas_a_borrar = ['country_residence', 'gender']
X_train_tree.drop(columns=columnas_a_borrar, inplace=True)
X_test_tree.drop(columns=columnas_a_borrar, inplace=True)
X_val_tree.drop(columns=columnas_a_borrar, inplace=True)

modelo_clasificacion_arbol = DecisionTreeClassifier(
    class_weight='balanced',
    max_depth=7,
    random_state=42
)

modelo_clasificacion_arbol.fit(X_train_tree, y_train)
```

```

predicciones_arbol = modelo_clasificacion_arbol.predict(X_test_tree)

tabla_arbol = pd.crosstab(
    y_test, predicciones_arbol,
    rownames=['Realidad (0=No, 1=Sí)'], colnames=['Predicción (0=No, 1=Sí)']
)

acc_arbol = accuracy_score(y_test, predicciones_arbol)
rec_arbol = recall_score(y_test, predicciones_arbol, pos_label=1)

print("=== MATRIZ DE CONFUSIÓN (ÁRBOL BALANCEADO) ===\n")
print(tabla_arbol)
print("\n=== EVALUACIÓN ===")
print(f"Accuracy Global: {acc_arbol:.2%}")
print(f"Recall (Detección del Muro): {rec_arbol:.2%}")

```

=== MATRIZ DE CONFUSIÓN (ÁRBOL BALANCEADO) ===

Predicción (0=No, 1=Sí)	0	1
Realidad (0=No, 1=Sí)		
0	7699	4914
1	888	2307

=== EVALUACIÓN ===

Accuracy Global:	63.30%
Recall (Detección del Muro):	72.21%

Este modelo, en comparación con el anterior, tiene mucha más *Recall* pero menos *Accuracy*.

4.3.4 Random Forest

(hacer el modelo)

4.3.5 Selección del mejor modelo y predicción en el conjunto de Validación

Tras haber entrenado y probado en test los 3 modelos anteriores, el que obtiene mejores resultados es el modelo de XXXXXXXX

Por ello, se realizan las predicciones en el conjunto de validation:

```
#1. Elegimos el modelo entre los tres, modelo_clasificacion_arbol, rforest o knn
predicciones_validacion = modelo_clasificacion_arbol.predict(X_val_tree)

tabla_arbol = pd.crosstab(
    y_test, predicciones_validacion,
    rownames=['Realidad (0=No, 1=Sí)'], colnames=['Predicción (0=No, 1=Sí)']
)

acc_arbol = accuracy_score(y_test, predicciones_validacion)
rec_arbol = recall_score(y_test, predicciones_validacion, pos_label=1)

print("=== MATRIZ DE CONFUSIÓN (ÁRBOL BALANCEADO) ===\n")
print(tabla_arbol)
print("\n=== EVALUACIÓN ===")
print(f"Accuracy Global: {acc_arbol:.2%}")
print(f"Recall (Detección del Muro): {rec_arbol:.2%}")
```

```
=== MATRIZ DE CONFUSIÓN (ÁRBOL BALANCEADO) ===
```

Predicción (0=No, 1=Sí)	0	1
Realidad (0=No, 1=Sí)		
0	5353	7260
1	1313	1882

```
=== EVALUACIÓN ===
```

Accuracy Global:	45.77%
Recall (Detección del Muro):	58.90%

4.4 Problema de regresión. Predicción de tiempo final según sus datos hasta el parcial 35km

El objetivo de esta sección es predecir los tiempos finales de los corredores con los datos hasta el parcial de 35km. Por lo tanto,